Metodologia de Filtragem e Classificação por Nome de Padrão/Plataforma

GitSkills — Segurança de Smart Contracts em Agent Skills

Pamella Soares

4 de setembro de 2026 (v4)

1. Linhagem de dados — do .db original a esta abordagem

Passo	Ação	Entrada → Saída	Justificativa
0	Coleta bruta GitSkills (jul/2026)	GitHub code search → **41 GB**, 3.797.117 arquivos / 1.877.981 skills distintas	Universo completo de `SKILL.md` públicos.
1	Filtro por vocabulário genérico de blockchain (389 `mandatory_terms`)	41 GB → `agent_skills_mandatory_only.db`, **89.649 skills**	Rede de alto *recall*/baixo custo antes de filtros específicos.
2	Filtro por **nome próprio** de check-list/ferramenta/firma (`scripts/01_keyword_filter.py`)	89.649 → **1.592 skills**	Nomes próprios (Slither, SCSVS, Trail of Bits…) são sinal de precisão muito mais alto que termos genéricos de segurança.
3	Classificação por skill (LLM, evidência verbatim) — `scripts/02_llm_classifier.py`	1.592 → registro estruturado por skill	Ver Seções 3–5.
4	Duplicação e proliferação de template — `scripts/04_template_network.py`	1.592 → 151 clusters de quase-duplicata	Ver Seção 6.

Passo	Ação	Entrada → Saída	Justificativa
5	Referências explícitas entre skills — `scripts/05_reference_network.py`	1.592 → grafo de citação	Ver Seção 7.
6	Visualizações quantitativas — `scripts/03_generate_plots.py`	JSON classificado → gráficos	Ver Seção 8.

A base `agent_skills_blockchain_filtered.db` (21.827 skills, filtro por nome de repositório/organização conhecida) não faz parte desta linhagem — é insumo de outra abordagem do projeto.

2. Filtro por nome de padrão/plataforma

2.1 Lista de termos (46 no total)

41 termos fortes — contam sozinhos:

Categoria	Termos
Checklists/padrões	SCSVS, SWC Registry, SWC-1, OWASP Smart Contract, DASP, ConsenSys, ConsenSys Diligence, Ethereum Smart Contract Best Practices, building-secure-contracts, building secure contracts, not-so-smart-contracts, secure development workflow, Veridise Security Checklist
Ferramentas/scanners	Slither, Mythril, Echidna, Medusa, Manticore, Oyente, Rattle, Securify, solidity-coverage, eth-security-toolbox
Firmas/plataformas	Trail of Bits, Certik, Halborn, PeckShield, SlowMist, Zellic, Quantstamp, Hacken, Cyfrin, Solodit, Sherlock, Code4rena, Immunefi, Spearbit, Sigma Prime, Ackee Blockchain, Least Authority, Dedaub

5 termos fracos (só contam combinados com um termo forte): OWASP, OWASP Top 10, OpenZeppelin, CWE-, STRIDE.

2 termos descartados por ambiguidade: `SWC-` isolado (colide com "react-swc") e `runtime verification` (frase genérica).

2.2 Resultado e teste de saturação

1.592 skills passam no filtro (`data/checklist_platform_filter/checklist_platform_candidates_v3_15`
Testados 4 termos candidatos adicionais contra a base completa:

Termo candidato	Hits	Decisão
`known attacks`	20	Descartado — ruído de segurança de IA/prompt-injection
`severity classification system`	7	Descartado — code review genérico
`ethtrust`	5	Descartado — 100% redundante com outros termos
`immunefi vulnerability severity`	0	Descartado

Nenhum termo alterou o pool. **1.592 é um resultado estável**, não um corte arbitrário.

3. Schema de classificação por skill

Cada skill recebe: `nature` (nível da skill) + `tools` / `firms_platforms` / `checklists` (por item nomeado citado) + `vulnerabilities` (lista simples). Evidência é sempre citação verbatim, verificada programaticamente como substring literal do texto-fonte (normalizando espaço/quebra-de-linha de YAML e decoração markdown como crases e pipes de tabela).

3.1 Natureza (nível da skill) — 4 categorias

Valor	Significado
`follows_standard`	Modela a orientação de segurança explicitamente sobre um padrão/checklist externo nomeado
`own_standard`	Organiza processo/checklist próprio, estruturado, sem se apoiar num padrão externo nomeado
`audit_support_tool`	Pertence ao entorno de uma auditoria (formata relatório, consulta banco de achados históricos) mas **não executa** análise de vulnerabilidade
`not_security_review`	Não é de fato conteúdo de revisão de segurança de smart contract (falso positivo do filtro)

A categoria `ad_hoc` (do desenho original) foi **removida**: nenhuma das skills testadas se encaixou nela, e a fronteira entre "disperso" e `own_standard` era ambígua sem caso real de uso.

Regra de curto-circuito: se `nature` for `not_security_review` ou `audit_support_tool`, `tools`/`firms_platforms`/`checklists` ficam vazios. Essa regra é **aplicada em código**

(enforce_short_circuit em 02_llm_classifier.py), não apenas pedida no prompt — o modelo às vezes preenchia esses blocos mesmo devendo deixá-los vazios.

3.2 Referências externas (por item nomeado)

- **tools** — papel: recommended_run / output_consumed / embedded_ruleset / mentioned_reference.
- **firms_platforms** — papel: methodology_source / data_source / library_dependency / named_reference.
- **checklists** — papel: primary_structure / partial_reference / named_only.
- **Nível de entrega** (points_externally): embedded / internal_pointer / external_pointer / mixed.

3.3 Vulnerabilidades mencionadas (campo novo)

Lista simples (sem papel/evidência) das categorias de vulnerabilidade citadas no texto (ex.: "Reentrancy", "Access Control", "Oracle Manipulation"). Normalizada em Title Case na agregação — o LLM não é consistente em capitalização ("Reentrancy" vs. "reentrancy" apareciam como categorias separadas antes da correção).

4. Prompt de classificação

Template completo em scripts/02_llm_classifier.py (PROMPT_TEMPLATE). Pontos reforçados após os primeiros testes:

- **Proibição explícita de reticências** ("…", "[…]") para abreviar citação longa — o modelo tentava resumir em vez de citar literalmente.
- **Proibição de reconstruir tabelas markdown** — o modelo misturava cabeçalho com célula de dado, ou reordenava colunas, produzindo uma paráfrase "legível" em vez de um trecho contíguo real.
- Campo vulnerabilities (Seção 3.3).

Verificação pós-hoc: toda *_evidence precisa ser substring literal (case-insensitive, espaço/quebra-de-linha normalizados, decoração markdown removida) do content — senão o item é reenviado com a lista de citações que falharam, até --max-retries.

5. Execução em escala

5.1 Modelo

mistral-medium-latest/mistral-small-latest/magistral-* retornam 429 rate_limited **nesta conta** mesmo em requisição isolada e após espera — não é rajada, é restrição de tier por modelo. Testados todos os modelos disponíveis via /v1/models; os únicos liberados são a família ministral-*, open-mistral-nemo e codestral. Default do script: ministral-14b-latest (o maior liberado).

5.2 Checkpoint incremental

Rodar as 1.592 skills leva ~1-2h. O script grava **uma linha JSON por skill imediatamente após classificá-la** (<output>.checkpoint.jsonl, com flush() a cada linha)

e, ao reiniciar, pula automaticamente quem já está no checkpoint (`load_checkpoint` / filtro de `pending`). Perda máxima em caso de interrupção: 1 item em andamento.

5.3 Rotação de chaves

Qualquer variável `MISTRAL_*` em `.env` é usada; em 429/401/403 o `KeyRotator` avança para a próxima chave automaticamente.

Comando final

```
cd scripts
python3 -u 02_llm_classifier.py \
  --sample-size 1592 \
  --output ../data/checklist_platform_filter/llm_classification_full.json \
  --sleep 1.5
```

6. Validação

Amostra de **10 skills** diversas, classificação manual vs. LLM (`ministral-14b-latest`): **10/10 evidências verbatim** após os fixes do prompt, **9/10 de concordância** em `nature`. A única divergência (`web3-testing`) é defensável — o modelo tratou um guia de testes genérico como "metodologia de teste de segurança própria", onde a leitura manual foi "sem conteúdo de vulnerabilidade". Referência: `data/checklist_platform_filter/test10_merged_example_claude.json` (manual) vs. `test10_llm_classification.json` (LLM).

7. Duplicação e proliferação de template

Achado principal: `scripts/04_template_network.py` detecta quase-duplicatas via MinHash/LSH (shingles de 8 palavras sobre o `content` já limpo de HTML/URLs/badges) — escala para o pool inteiro sem custo de API.

- **688 repos distintos / 664 owners distintos** para 1.592 skills — concentração moderada; maior owner (`tools-only`) contribui 103 skills (6,5% do pool).
- **151 clusters de quase-duplicata** (similaridade ≥ 0,8), envolvendo **694/1.592 skills (43,6% do pool)** — quase metade das "skills independentes" do pool são cópias/variações de um template comum.
- Maiores clusters: `web3-testing` (36 cópias, 35 owners distintos), `blockchain-developer` (33 cópias, 32 owners), `engineering-solidity-smart-contract-engineer` (20, 19 owners), `blockchain-security-auditor` (20, 19 owners), `bug-bounty` (18, 14 owners).
- Alguns clusters são duplicação **dentro do mesmo owner** (ex.: 23 itens de `tools-only`), não propagação entre pessoas.

Implicação metodológica: contagens brutas (tópico, keyword, `nature`) ficam infladas por template. Recomenda-se sempre reportar a versão bruta **e** a deduplicada por cluster (1 representante por família) nas análises subsequentes.

Saída: `data/checklist_platform_filter/network/` (`owner_concentration.png`, `template_clusters.json`, `template_clusters.png`, `template_network.png`).

8. Rede de referências explícitas entre skills

Diferente da Seção 7 (similaridade de texto = clonagem), `scripts/05_reference_network.py` mede **citação de propósito**: campo `related_skills` do frontmatter YAML e links markdown para outro `SKILL.md`, resolvidos como paths relativos dentro do mesmo repo.

- **36 referências resolvidas dentro do pool**, envolvendo 34 skills.
- **335 referências "dangling"** — apontam para uma skill-irmã (ex.: `methodology/SKILL.md`) que não bateu em nenhum termo forte sozinha e por isso não está no pool de 1.592.
- Rede esparsa mas real: pequenos clusters isolados por repo (composição interna de toolkits como `0x-Shashi/WEB3-AUDIT-SKILLS`), sem sobreposição entre owners diferentes — ao contrário da Seção 7, aqui não há propagação cross-owner.

Saída: `reference_edges.json`, `reference_network.png`, `reference_hubs.png` (skills mais referenciadas = "hubs" de metodologia dentro de um toolkit).

9. Visualizações quantitativas

`scripts/03_generate_plots.py`, a partir do JSON classificado:

- `nature.png` — distribuição da categoria principal (inclui categoria "falhou (erro na API)" para itens sem resposta válida, em vez de quebrar no `None`).
- `tools_named.png` / `firms_platforms_named.png` / `checklists_named.png` — barra horizontal empilhada: nome citado × papel (ex.: Slither aparece 2× como `mentioned_reference`, 1× como `recommended_run`).
- `points_externally.png` — nível de entrega agregado dos três blocos.
- `vulnerabilities.png` — top categorias de vulnerabilidade mencionadas, normalizado por Title Case.
- `wordcloud_pool.png` — nuvem de palavras do `content` de todo o pool (1.592), com bigramas (`collocations=True`) e limpeza de ruído estrutural (URLs, badges markdown, tags/entidades HTML) e de meta-vocabulário de tooling (Claude Code, slash command, workflow...) via `scripts/text_cleaning.py`, módulo compartilhado com o script 04.

10. Limitações conhecidas e próximo passo

Confiabilidade de conteúdo (não implementado ainda): os testes de duplicação/rede não avaliam se o conteúdo propagado está **correto**. Abordagem recomendada, em duas camadas:

1. **Checagem determinística** (sem risco de alucinação): IDs `SWC-XXX` citados vs. registro SWC real; nomes de ferramentas/firmas vs. as listas já mantidas em `01_keyword_filter.py`; comportamento de versão de Solidity citado vs. real.
2. **Revisão por LLM, separada e priorizada**: prompt à parte (não misturado ao de classificação), aplicado apenas aos representantes dos ~20-30 maiores clusters de template (Seção 7) — um erro ali afeta dezenas de repos de uma vez, então revisar 1 vez por cluster é mais eficiente e mais impactante do que revisar as

1.592 individualmente. Resultado tratado como candidato para revisão humana, não como verdade — o próprio verificador pode errar.

Estado da execução em escala: em andamento; ver `llm_classification_full.checkpoint.jsonl` para contagem corrente de itens processados.